DÍA MES AÑO NOTA /

Sistemas Digitales

Estudiante: Emilio José Gálvez Merchán.

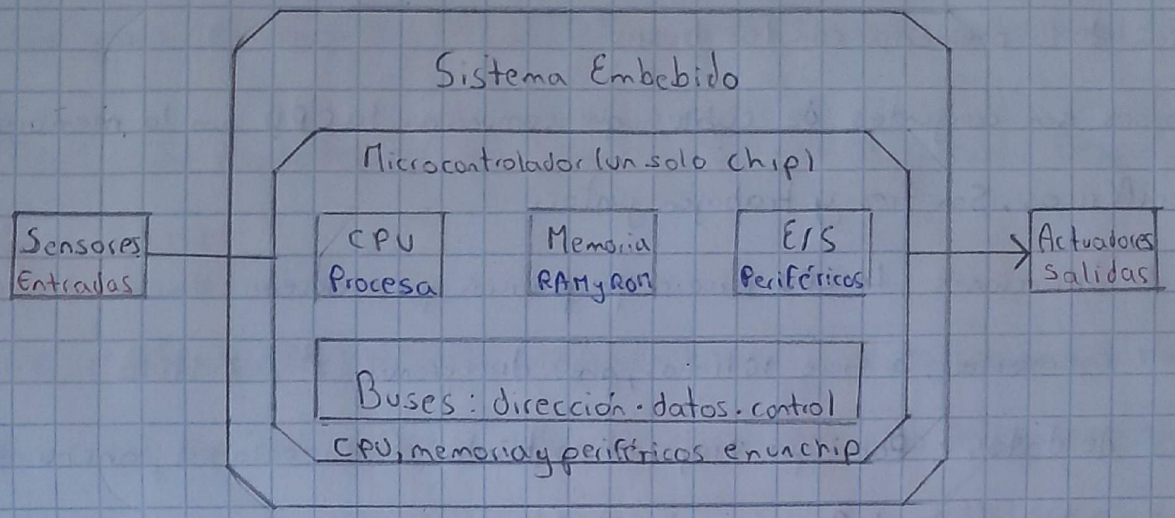
Ciclo: 5to Ciclo

Docente: Ing. Luis Darío Sinche Cueva, Mg. Sc.

Carrera: Computación

Escaneado con CamScanner

Sistema Embebido: Es un computador dedicado a una sola tarea dentro de un aparato: lavadora, microondas, el control de un carro



Microprocesador vs microcontrolador

- Microprocesador (μP): Sola CPU, necesita componentes externos: memoria RAM, memoria para el programa, puertos de entrada/salida, etc.

  Ejemplos: El intel 8086 y los Pentium/Core → flexible y potente, pero más caro y grande. Se usa en PCs.

- Microcontrolador (μC): Es un computador completo en un solo chip. Trae dentro la CPU, la memoria (Flash y RAM), los puertos de entrada/salida, los timers, el ADC y la comunicación serial.

  Ejemplos: El 8051, el ATmega328 (Arduino UNO) y el STM32. Es barato de bajo consumo → Controlar cosas, se usa en sistemas embebidos.

¿Qué es la memoria?

Es donde se guardan bits, y se divide en dos tipos de información

1. Instrucciones: Programa que la CPU debe ejecutar.
2. Datos: Son las variables, lecturas de sensores y resultados.

Escaneado con CamScanner

Cada posición de memoria tiene una dirección y guarda un valor, normalmente de 8 bits (1 byte)

Relación de la memoria con los buses

Los buses son conjuntos de cables que comunican la CPU con la memoria y los periféricos. Son tres y trabajan juntos:

1. Bus de direcciones: La CPU dice a que posición quiere acceder: Un solo sentido de CPU → memoria. Si tiene 16 líneas, puede direccionar $2^{16} = 64KB$
2. Bus de datos: Por aquí viaja el contenido. Bidireccional, ancho de (8, 16, 32 bits), define cuantos bits se mueven a la vez.
3. Bus de control: Señales que dicen que operación hacer: Lectura (RD), escritura (WR) o chip select (CS).

Ejemplo: Leer la posición 0x1000: la CPU pone 0x1000 en el bus de direcciones, activa RD en el de control, y la memoria responde poniendo el dato en el bus de datos. Para escribir es igual, pero activa WR y es la CPU quien pone el dato en el bus

Clasificación de las memorias

- Volátiles: Se borran sin energía → RAM
- No volátiles: Conservan el contenido sin energía → ROM, EPROM, EEPROM y Flash

-RAM: Memoria de trabajo donde se leen y escriben datos rápidamente. Guarda variables, la pila y datos temporales. Hay SRAM (rápida, mas cara, usada en microcontroladores y cachés) y DRAM (más densa, usada en la memoria principal de PCs)

Escaneado con CamScanner

- ROM (Read Only Memory): Su contenido se guarda en la fábrica y ya no se puede cambiar. Solo se lee (Barata en grandes volúmenes, pero poco flexible) Se usó para el arranque (El BIOS original) y para programas fijos

- PROM: Se puede reprogramar una sola vez por el usuario. Después queda como una ROM.

- EPROM (Erasable PROM): Se puede borrar y reprogramar. El borrado se hace con luz ultravioleta a través de una ventanita de cuarzo que tiene el chip (borra todo el contenido) a la vez. Era incómoda → Había que sacar el chip del circuito.

- EEPROM (Electrically Erasable PROM): Se borra y reescribe electricamente, sin sacarla del circuito, y se hace byte por byte. Sirve para guardar datos de configuración que cambian poco (calibraciones, ajustes). Su límite es la cantidad de ciclos de escritura (del orden de 100mil a un millón) y que es más lenta.

- Flash: Es una evolución de la EEPROM. Se borra electricamente, pero por bloques o sectores en lugar de byte por byte, y eso la hace más densa, más barata y más rápida. Es la memoria donde se guarda el programa en los microcontroladores actuales, como los 512 KB de Flash del STM32F446. También es la memoria de las USB y SSD. Se puede reprogramar miles de veces.

ROM → PROM → EPROM → EEPROM → Flash
(fija) (1vez) (UV) (byte) (por bloques)

Escaneado con CamScanner

DIA MES AÑO NOTA /

Sistema binario: Sistema de numeración 2, ya que usa 0 y 1. Se usa porque los circuitos digitales trabajan con dos estados (transistor apagado/encendido, voltaje bajo/alto). 0 y 1 es lo que el hardware entiende

Como Funciona

Cada posición vale una potencia de 2

Posición	2^3	2^2	2^1	2^0
Peso	8	4	2	1
Bit	1	0	1	1

Se suman los pesos donde hay un uno: $8+2+1=11$

De decimal a binario

Para pasar de decimal a binario se divide entre 2 y se leen los residuos de abajo hacia arriba.

13 | 2
1 6 | 2
0 3 | 2
1 1

Rta: 1101

Sistema Hexadecimal: Es de base 16, en donde se necesita 16 simbolos del 0 al 9 y luego A, B, C, D, E, F (que valen 10 a 15)

Decimal	0	1	2	3	4	5	6	7	8
Hexadecimal	0	1	2	3	4	5	6	7	8
Binario	0000	0001	0010	0011	0100	0101	0110	0111	1000

Decimal	9	10	11	12	13	14	15
Hexadecimal	9	A	B	C	D	E	F
Binario	1001	1010	1011	1100	1101	1110	1111

La clave es que un digito hexadecimal equivale exactamente a 4 bits. Por eso un byte se escribe con solo 2 digitos hex: 1010 1111 binario
A F Hexadecimal → 0xAF

Escaneado con CamScanner

Para pasarlo a decimal: 0xAF = 10 x 16 + 15 = 175

Para que se usa cada uno

Binario: Es el lenguaje real de la máquina. Se usa para pensar en bits individuales: máscaras, banderas, estado de pines, operaciones lógic-as (AND, OR, XOR). Es largo y difícil de leer para humanos.

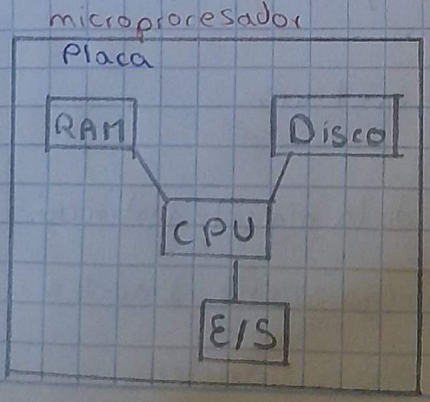
Hexadecimal: Forma compacta de escribir binario. Se usa para direcciones de memoria, código máquina, valores de registros y volcados de memoria

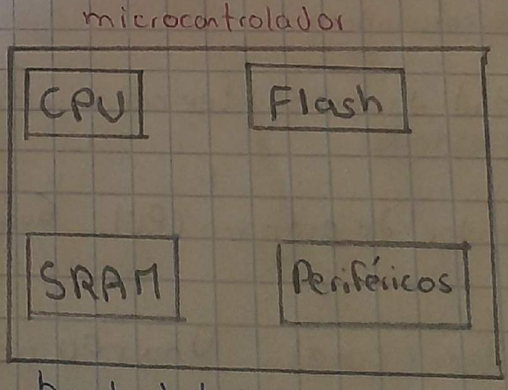
Comparando 16 bits: En binario 1010 1011 1100 1101, en hexadecimal 0xABCD.

CPU: Unidad central de procesamiento

microprocesador = procesador -> Lo tienen computadoras, celulares
↓
Cerebro de la máquina

Dentro de un sistema embebido -> Realiza una tarea específica
↓
hay un microcontrolador
-> Recursos limitados
-> Poca memoria
-> Poco consumo
-> Funciona sin que el usuario lo note.
-> Muchas veces debe responder a tiempo



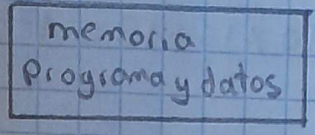


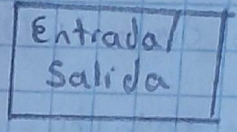
barato de bajo consumo

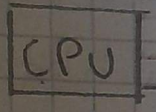
Escaneado con CamScanner

DIA MES AÑO NOTA /

Computador







ejecuta el programa

- Bus de direcciones. ¿Que casillero?, a donde va la información
- Bus de datos: El contenido, la información en sí.
- Bus de control: Operaciones, leer o escribir, otras

• Un flip-flop es un circuito digital secuencial que almacena 1bit: tiene dos estados estables, SET(1) y RESET(0), y los mantiene indefinidamente hasta que una entrada o el reloj los cambia. Su salida depende de la entrada actual y del estado anterior.

Cuanto cabe KB y MB		
1 byte	1KB	1MB
8bits	1024	1024KB

ATmega 328P		
Flash	32KB	4mB
SRAM	2KB	520KB
EEPROM	1KB	No tiene (se emula)

Se almacena información a través de capacitores

Dentro de la CPU

ALU: Unidad aritmetica-lógica: Suma, resta y hace AND, OR, XOR
Supercalculadora

Unidad de control: Lee cada instrucción y decide que hace.
orquestador

Contador de programa (PC): Guarda la dirección de la siguiente instrucción

Nucleo (core): Una CPU completa

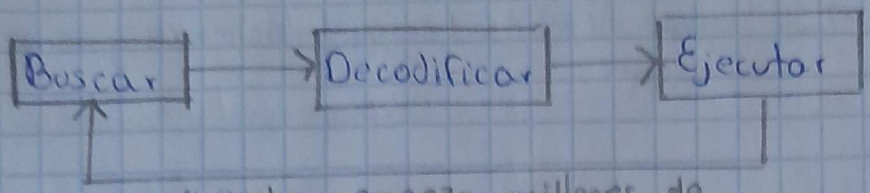
ATmega 328P: 1 núcleo

EsP32: 2 núcleos

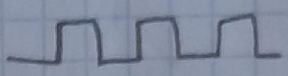
Escaneado con CamScanner

DÍA MES AÑO NOTA /

La CPU repite tres pasos al ritmo del reloj



El reloj



16 MHZ

16 millones de pulsos por segundo

Buscar: Traer la instrucción desde la memoria

Decodificar: Entender que pide ¿Que necesita?

Ejecutar: Hacerlo

Bits, operaciones bit a bit y E/S mapeada en memoria

Como transformar

- Binario → Decimal

1101 = 13 → Sumar las potencias 2 donde hay un 1.

$1 \cdot 2^0 = 1$
$0 \cdot 2^1 = 0$
$1 \cdot 2^2 = 4$
$1 \cdot 2^3 = 8$
13

- Decimal a Binario

13 = 1101 → Dividir entre dos y leer los residuos de abajo hacia arriba.

13 | 2
① 6 | 2
0 3 | 2
① 1

Escaneado con CamScanner

- Binario → Hexadecimal

1011 1101 = BD → Se agrupa de 4bits (desde la derecha) y convierte cada grupo

1011 → 11 → B
1101 → 13 → D

en Hexadecimal

- Hexadecimal → Binario

3F = 0011 1111 → Cada dígito pasa a 4bits

3 → 0011
F → 15 = 1111

15 | 2
1 7 | 2
1 3 | 2
1 1

- Hexadecimal → Decimal

2A = 42 → Se multiplica cada dígito por potencias de 16

$2 \cdot 16 + 10 = 42$ → $2 \cdot 16^1 + 10 \cdot 16^0 = 32 + 10 \cdot (1) = 42$

- Decimal → Hexadecimal

255 = FF → Se divide entre 16 y lee los residuos al reves

255 | 16
95 | 15
15 | F
F

Cuando tenemos 0x, es solo un prefijo que indica "este número está en hexadecimal", no forma parte del valor. Así que 0x20 = 20

Ejemplo: 0X20 = $2 \cdot 16 + 0 = 32$ decimal
0X20 = 2 + 0 = 00100000 binario
0010 + 0000

Escaneado con CamScanner

Convertir

1000 0001 = 129 decimal y 0x81 hexadecimal

a decimal y Hexadecimal →

↓	↓	
$1 \cdot 2^0 = 1$	1000	0001
$1 \cdot 2^7 = 128$	↓	↓
129	8	1

0x FF = 1111 1111 binario y 255 decimal

a binario y a decimal

↓	↓
F F	$15 \cdot 16^1 + 15 \cdot 16^0 = 240 + 15 = 255$
↓ ↓	
15 15	

15 |2
(1) 7 |2
(1) 3 |2
(1) (1)

0x08 = el bit 3

¿ que bit esta en 1?

0000 1000

200 = 11001000 binario y 0xC8 hexadecimal

a binario y a hexadecimal →

200 |2
(0) 100 |2
(0) 50 |2
(0) 25 |2
(1) 12 |2
(0) 6 |2
(0) 3 |2
(1) (1)

200 |16
40 (12 → C) 8
(08)

Escaneado con CamScanner

Las compuertas lógicas, un bit a la vez

A	B	AND &	ORI |	XOR ^	NOTA ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

AND: 1 solo si ambos son 1

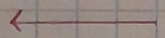
OR: 1 si alguno es 1

XOR: 1 si son distintos

NOT: Invierte

La máscara: 1 << n señala un bit

0 0 0 0 0 0 0 (1) 1
7 6 5 4 3 2 1 0



correr s lugares a la izquierda

0 0 (1) 0 0 0 0 0 1 << 5 = 0x20

Una máscara es un byte con 1 solo en los bits que quiero tocar.

Escribir 1<<5 es más legible que 0x20 dice el "bit 5"

Las cuatro recetas

Quiero	En C	Por qué funciona
Bit n en 1	REG |= (1<<n);	OR con 1 da 1, OR con 0 deja igual
Bit n en 0	REG &= ~(1<<n);	~ invierte la máscara; AND con 0 da 0
Invertir bit n	REG ^= (1<<n);	XOR con 1 invierte, XOR con 0 deja igual

Preguntar por if (REG & (1<<n)) Queda distinto de 0 solo si el bit está en 1
bit n

Escaneado con CamScanner

Ejercicios

REG = 10101010 Cada ejercicio parte de este valor

1. REG1 = (1<<0);

```
mascara          -> 10101010   OR
                    00000001
00000001            10101011 ,,
```

2. REG8 = ~(1<<7);

```
mascara                          10101010  AND
                ~                01111111
10000000 -> 01111111  -> 00101010 ,,
```

3. REG ^= 0x0F;

```
                          -> 10101010   XOR
0x0F -> 0000 1111         -> 00001111
   |       mascara         10100101 ,,
   v
  1S
```

4. ¿Se cumple ¡F(REG & (1<<3))?

```
mascara                   10101010  AND
                -> 00001000
 00001000         00001000   -> TRUE
```

Juego de instrucciones del AVR

Instrucción	Qué hace
LDI r16, 0x20	Carga el número 0x20 en el registro r16
ADD r16, r17	Suma r17 a r16
AND OR EOR COM	AND, OR, XOR y NOT bit a bit las instrucciones lógicas
SBI 0x05, 5	Pone en 1 el bit 5 del registro de E/S 0x05 (PORTB)
CBI 0x05, 5	Pone en 0 ese mismo bit
RJMP	Salta a otra parte del programa.

Escaneado con CamScanner

Demostración: de C a bits

Lenguaje C: `PORTB |= (1<<5);`

↓

Ensamblador: `sbi 0x05, 5`

↓

Código maquina en la Flash: 1001 1010 0010 1101 (0x9A2D)

RISC y CISC; 8 y 32 bits

RISC

Pocas instrucciones, simples; casi todas en un ciclo de reloj.

AVR, ARM, RISC-V, Xtensa (ESP32).

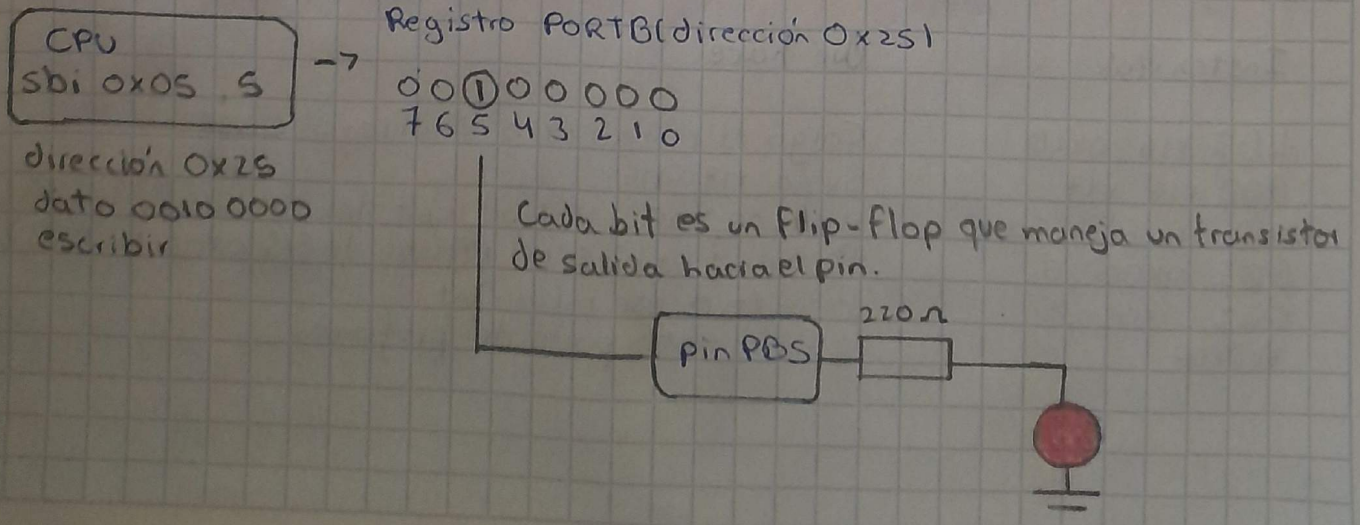
CISC

Muchas instrucciones, algunas muy elaboradas

x86, el de los PC

8bits o 32 bits: Cuántos bits procesa la CPU de una vez. El ATmega328P suma 8 bits por instrucción; para sumar números de 32 bits necesita 4 sumas. El ESP32 lo hace en una.

La idea central: escribir un bit enciende un LED



Escaneado con CamScanner

Registros por puerto

Registro	Para qué sirve	Bit en 1 significa
DDRB	Dirección: entrada o salida	Salida
PORTB	Salida: el valor que sale. Entrada (pull up)	5 V en el pin, o pull up activa
PINB	Leer el voltaje real del pin	El pin está en alto

El bit n de cada registro corresponde al pin n del puerto. El ATmega 328P tiene los puertos B, C y D.

Cuatro estados de un pin

DDRxn	PORTXn	Estado del pin	Ejemplo
0	0	Entrada flotante: lee al azar	Nada conectado
0	1	Entrada con pull-up	Botón a GND
1	0	Salida en 0 (0V)	LED apagado
1	1	Salida en 1 (5V)	LED encendido

Con pull-up, el botón suelto lee 1 y pulsado lee 0: la lógica queda invertida.

Predice el pin

1 DDRB5 = 1 PORTB5 = 1
¿Voltaje en el pin?
5V

2 DDRB5 = 1 PORTB5 = 0
¿Voltaje en el pin?
0V

3 DDRD2 = 0 PORTD2 = 1
Botón pulsado: ¿PIND2?
Lee 0

4 DDRD2 = 0 PORTD2 = 0
Nada conectado. ¿PIND2?
Indeterminado (Puede leer 0 o 1 por ruido)

Escaneado con CamScanner

Funciones con arduino

Con funciones de Arduino	Con registros (lo que pasa por dentro)
`pinMode(13, OUTPUT);`	`DDRB |= (1<<5);` → Poner 1 lógico en el pin 5 para que se comporte como salida
`digitalWrite(13, HIGH);`	`PORTB |= (1<<5);` enciende
`digitalWrite(13, LOW);`	`PORTB &= ~(1<<5);` apaga
`pinMode(2, INPUT_PULLUP);`	`DDRD &= ~(1<<2);` PD2 como entrada
	`PORTD |= (1<<2);` Activa pull up
`int b = digitalRead(2);`	`int b = (PIND & (1<<2)) != 0;`

Las funciones son más legibles y portables, los registros son más rapidos y muestran qué ocurre en el chip.

Ejercicios

¿Cuanto es 0x81 en binario?

8 = 1000 Rta = 10000001

1 = 0001

¿Escribe la línea que apaga el bit 4 PORTB?

`PORTB &= ~(1<<4);`

Con PORD2 = 0 y PORTD2 = 1, ¿qué lee PIND2 con el boton pulsado a GND?

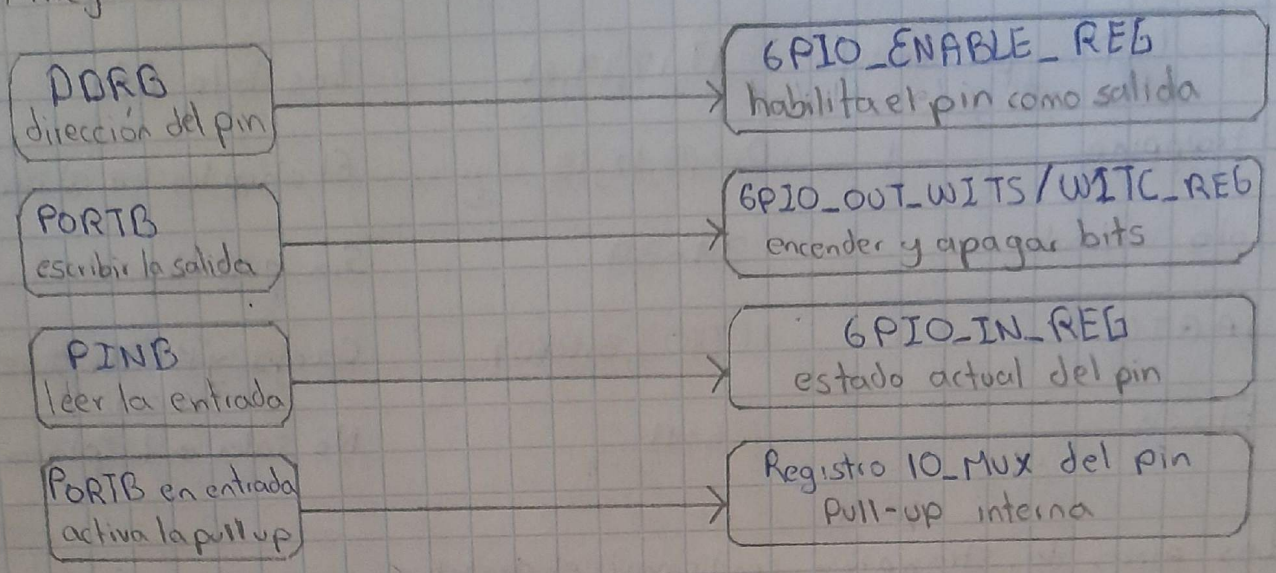
Lee 0 V

Escaneado con CamScanner

Del AVR al ESP 32

ATmega 328P (AVR)

ESP32



Por qué existen W1TS y W1TC

En el AVR, PORTB |= (1<<PB0) hace tres pasos: leer, modificar y escribir. Si una interrupción cambia el registro entre la lectura y la escritura, ese cambio se pierde.

Con dos núcleos el riesgo es mayor.

El ESP32 lo resuelve con dos registros que solo escriben, sin leer antes

```
GPIO_OUT antes:  0 1 0 1
escribo en W1TS: 0 0 1 0   (1 = encender, 0 = no tocar)
GPIO_OUT después: 0 1 1 1
```

W1TS (write 1 to set) enciende los bits en 1. W1TC (write 1 to clear) apaga los bits en 1. Como es una sola escritura, la operación es atómica.

Capas de abstracción

digitalWrite() llama al core Arduino-ESP32, que llama a ESP-IDF (gpio_set_level), que escribe el registro y el pin sube a 3,3V. Cada capa te da más comodidad, pero también más tiempo de ejecución. Los registros dan más control y velocidad.

Escaneado con CamScanner

Pines del DevKit V1

Tipo	Pines	Nota
Seguros (LEDs y pulsadores)	16, 17, 18, 19, 21	Los de la AP2
Strapping	0, 2, 5, 12, 15	El chip los lee al arrancar
Solo entrada	34-39	Sin pull-up interna
UART0	1 (TX0), 3 (RX0)	Carga y Monitor Serie
Flash	6-11	No salen en la placa

Los pines de strapping condicionan el arranque:

- GPIO0 en bajo activa el modo de carga (es el botón BOOT)
- GPIO2 debe estar en bajo o al aire para cargar
- GPIO12 en alto al arrancar pone la flash a 1,8V y la placa no arranca.

Los límites eléctricos son estos:

- Los pines no toleran 5V. Un sensor de 5V conectado directo puede dañarlos
- Un LED rojo con 330Ω consume (3,3 - 2,01) / 330 ≈ 4mA, y el diseño recomienda ≤10mA por pin.
- Los LEDs azul y blanco casi no encienden a 3,3V.

Preguntas

A LED en GPIO18

Pin seguro

Pulsador en GPIO35 con INPUT_PULLUP

GPIO35 no tiene pull up interna

Escaneado con CamScanner

DÍA MES AÑO NOTA

LED en GPIO2

Es un pin de strapping

Pulsador entre GPIO12 y 3,3V

Deja GPIO en alto al arrancar y la placa no arranca.

Sensor de 5V en GPIO4

GPIO4 no tolera 5V

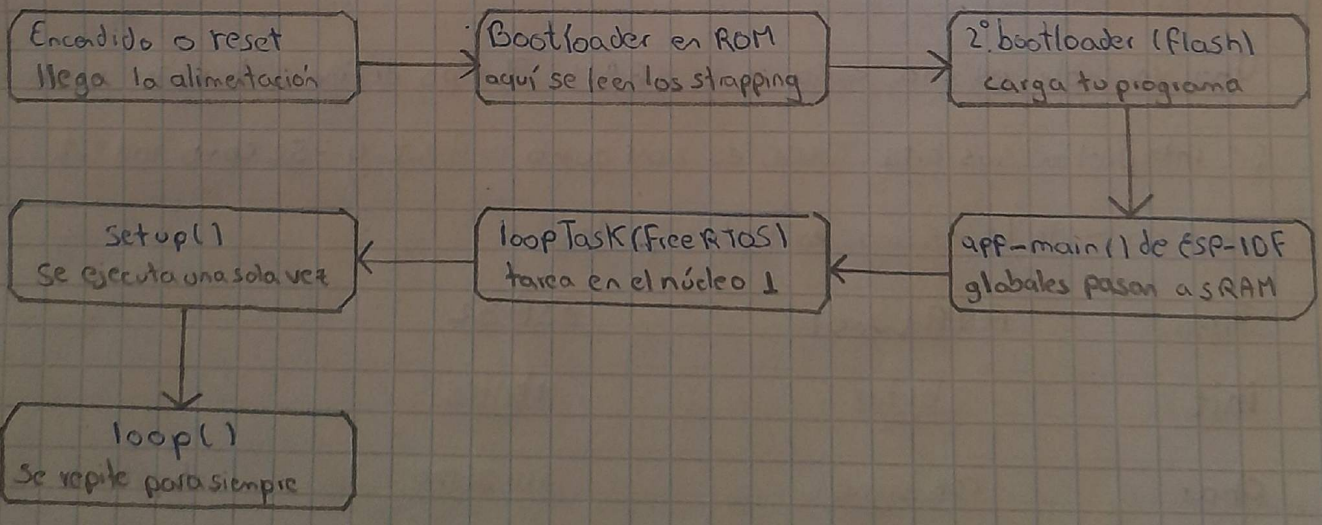
LED en GPIO1

Es TXO y interfiere con la carga y el Monitor Serie

El programa que nunca termina

Un programa de PC es como un visitante: lo lanza el sistema operativo, hace su trabajo y termina con return0. Un programa embebido es como un guardia nocturno: arranca con la alimentación, configura una vez y repite su ronda para siempre. En un microcontrolador no existe el "fin del programa"

Así llega el ESP32 desde el encendido hasta el loop():



Escaneado con CamScanner

Arduino esconde el superlazo, que en un AVR sería así:

```
int main() {
  init()          // timers, ADC...
  setup();
for(;;) {loop();}

}
```

En el ESP32, loopTask hace lo mismo

Predicción de la 3ª pasada del ejercicio:

- global = 3: la global conserva su valor
- local = 1: se recrea en cero en cada vuelta
- static = 3: conserva su valor entre llamadas, como la global

Un loop() vacío no sale gratis: la CPU gira a 240MHz y consume decenas de mA. En deep sleep consume del orden de uA, así que conviene preguntarse si el dispositivo necesita estar despierto todo el tiempo.

Programar para el hardware

Tipos de datos

Una variable es una caja en la SRAM. El tipo define su tamaño y cómo se interpretan sus bits: 0xCC es 204 como uint8_t y -52 como int8_t.

Int no mide lo mismo en todas partes

Tipo	AVR (Uno)	ESP32
int	2 bytes	4 bytes
char	con signo	sin signo
double	4 (= float)	8, lo trata (software)
puntero	2	4

Escaneado con CamScanner

DÍA MES AÑO

Sobre el desbordamiento

- Sin signo, da la vuelta (255+1 = 0) y su comportamiento está definido
- Con signo, el comportamiento es indefinido y no hay que confiar en él.
- int t = millis() falla a los ~32s en el Uno y a los ~24,8 días en el ESP32. La solución es uint32_t.

Para constantes

- #define es solo reemplazo de texto, sin tipo.
- const tiene tipo y está verificada
- constexpr se calcula al compilar y es la que usaremos
- enum sirve para estados con nombre, como los modos del juego.

volatile, static y la pila

Volatile avisa al compilador que el valor puede cambiar sin que el programa lo vea, por una interrupción o por el hardware. Ojo: no significa atómico. Static dentro de una función que la variable conserve su valor entre llamadas.

Las variables locales viven en la pila de loopTask, que tiene 8 KB. Si se declara un arreglo local grande, la desborda y el ESP32 se reinicia.

Código	Error	Corrección
uint8_t i < 256	Siempre es verdadero, bucle infinito	int o uint16_t
int t0 = millis()	Tipo con signo que desborda	uint32_t
1 << 31	El 1 es un int con signo	1UL << 31
int veces = 0 en loop()	Se reinicia en cada vuelta	Static int veces = 0

Escaneado con CamScanner

El tiempo sin bloquear

delay() es como un cocinero que se queda mirando el horno: mientras espera, el programa no hace nada más, y si presionas el botón en ese momento, la pulsación se pierde. millis() equivale a mirar el reloj de pared sin detenerte.

```
uint32_t ahora = millis()
if (ahora - tAnterior >= PERIODO) {
tAnterior += PERIODO;
  // hace algo
}
```

El patrón es leer el reloj, restar, actuar si ya toca y seguir con lo demás.

Restar funciona incluso cuando el contador da la vuelta. Con un reloj imaginario de 8 bits. tAnterior = 250 y ahora = 4

Forma	Que sucede	Resultado
ahora - tAnterior	4 - 250 = 10 (módulo 256)	Es el tiempo transcurrido
millis() > tAnterior + PERIODO	La suma da la vuelta	La condición se cumple antes de tiempo.

Leer un pulsador bien

Pull-up: Con INPUT_PULLUP: suelto = HIGH, presionado = LOW (el pulsador va entre el pin y GND)

Rebote: La señal oscila unos instantes antes de quedarse quieta

Flanco: Un toque humano dura ~ 200ms, o sea miles de vueltas de loop().

Escaneado con CamScanner